Regression Phenomena and Testing

Ram Prasad

5-21-2025

Markets PPNR

I demonstrate basic phenomenon related to spurious regression, random walks, and overfitting, in support of model eligibility assessments for Markets PPNR models. I also assess the criteria prescribed by Model Risk Management for regression models. The intended audience is 2LoD model validators.

```
#Standard imports
import pandas as pd
import numpy as np
import statsmodels.api as sm
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
import time
import numpy as np
import scipy as sp
import matplotlib.ticker as mtick
import statsmodels
import cx_Oracle


#ADF and LB Tests
from statsmodels.tsa.stattools import adfuller as adf
from statsmodels.stats.diagnostic import acorr_ljungbox

#Misc
from itertools import product
from sqlalchemy.engine import create_engine
from itertools import chain, combinations

#set seed
np.random.seed(388)


#Vectorized simulation of AR(1)
def sim_AR1(phi, mu= 0.0, N = 1000, T = 100, sigma_w = 1.0, burn_in=0.20):
    """
    Simulate N AR(1) processes, each of length T.
    Args:
        phi (float): AR(1) parameter, -1 <= phi <= 1
        drift (float): AR(1) drif parameters, default is 0.0
        N (int): number of AR(1) simulations
        T (int): length of simulation
        sigma_w (float): std. dev. of WN term
        burn_in (float): percentage of obs to use as burn in
    Returns:
        sims (np.array): N x T simulated AR(1) processes.
    Default to 20% burn in. Each row is a sim of length T.
    """
    #Check phi valid
    if np.abs(phi)>1:
        raise ValueError(f"Input phi {phi} invalid.")

    #Total sim length is 1.2*T
    sim_T = int((1.0 + burn_in)*T)

    #Create N X T matrix of WN
    e = np.random.normal(loc=0, scale=sigma_w**2, size=(N,sim_T))

    #Initialize empty array
    sims = np.zeros(shape=(N,sim_T))


    #Populate sims
    for t in range(1, sim_T):

        #Iteratively compute X(t) = mu + phi*X(t-1) + e(t)
        sims[:,t] = mu + phi*sims[:,t-1] + e[:,t]

    #Drop burn in obs
    return sims[:,-T:]

#Function for MacKinnons CV using reg surface. See Appendix pp.2 (2010) and Table 2 N=1
def adf_cv(T, beta_inf = -1.94100, beta_1 = -0.2686, beta_2 = -3.365, beta_3 = 31.223):
    """
    Implement MacKinnon (2010) response surface regression equation. See the Appendix,
    Args:
        T (int): sample size
        beta_inf (float): asmptotic CV
        beta_1 (float): coefficient on 1/T
        beta_2 (float): coefficient on 1/T^2
        beta_3 (float): coefficient on 1/T^3
    Return:
        est_cv (float): estimated critical value using formula below
    Est. CV = Beta_inf + Beta_1/T + Beta_2/(T^2) + Beta_3/(T^3)
    Default values follow those in Appendix, pp.2, for N=1 and Tau_nc, the variant corr
    to no constant. This case is suitable for simulated AR(1)s: X(t) = phi*X(t-1) + e(t
    """
    #Ensure T strictly positive
    if not (T >=1):
        raise ValueError("Sample size must be >=1.")

    #Compute estimated CV and return
    est_cv = beta_inf + beta_1/(T) + beta_2/(T**2) + beta_3/(T**3)
    return est_cv


# Load data & data cleanning and transformation
def init_sql_connection():
    """
    Create initial SQL connection.
    Source: Qihao Hu, qh41038
    Returns
    -------
    create engine : sqlalchemy.engine.base.Engine
        sql engine for fetching data from oracle base.
```

```
"""
DIALECT = 'oracle'
SQL_DRIVER = 'cx_oracle'
USERNAME = '[redacted]
PASSWORD = [redacted]
HOST = [redacted]
PORT = [redacted]
SERVICE = 'PPNR2MD'
ENGINE_PATH_WIN_AUTH = DIALECT + '+' + SQL_DRIVER + '://' + \
    USERNAME + ':' + PASSWORD +'@' + HOST + ':' + str(PORT) + \
    '/?service_name=' + SERVICE
return create_engine(ENGINE_PATH_WIN_AUTH)
```

1a. Overfitting and Spurious Regression

I first demonstrate the phenomenon of overfitting coupled with spurious regression using a random walk as the dependent variable and polynomial time trends, of increasing degree, as covariates. There are two components:

1) For a driftless random walk, time trends should exhibit no predictive power. However, with enough covariates, and therefore non-linearity in the model, it is possible to perfectly interpolate the dependent observations.

2) When the dependent and independent variables are non-stationary (in this case, a random walk is by definition unit root and time trend variables exhibit time-varying moments), conventional measures of goodness of fit and statistical significance are meaningless.

To get a sense of how few polynomial time trends are necessary to spuriously model the random walk, achieving respectable R^2 and a majority of p-values significant at 5%, I perform a simulation exercise. The procedure is as follows:

A. Simulate n random walks without drift, each of length T:

$$X_{1,t} = X_{1,t-1} + e_{1,t}$$
$$\vdots$$
$$X_{n,t} = X_{n,t-1} + e_{n,t}$$

for $\vec{e} \sim N(\vec{0}, I_{nxn})$.

B. Regress each random walk on a set of polynomial time trends, incrementally adding covariates of higher degree. For simulation index $i = 1...n$, and covariate index $p = 1...T-1$, fit models:

$$X_{i,t} = \beta_{0,i} + u_{i,t}$$
$$X_{i,t} = \beta_{0,i} + \beta_{1,i}t + u_{i,t}$$
$$\vdots$$
$$X_{i,t} = \beta_{0,i} + \beta_{1,i}t + \cdots + \beta_{p-1,i}t^{p-1} + u_{i,t}$$
$$X_{i,t} = \beta_{0,i} + \beta_{1,i}t + \cdots + \beta_{p-1,i}t^{p-1} + \beta_{p,i}t^{p} + u_{i,t}$$

C. For the fitted models associated with each simulated random walk, compute the MSE of the residuals, Adjusted R^2, and the proportion of parameter estimates significant at 5% for each.

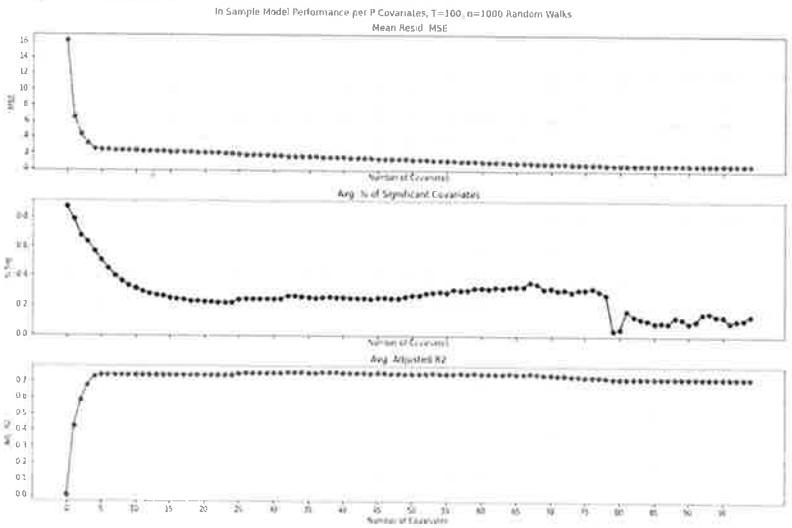
D. For each submodel (defined by the number of time trend covariates), average goodness of fit and significance measures across the n simulations.

Let $n = 1,000$ and $T = 100$.

```
#Time execution
t0 = time.time()

#Simulate RW of length T, no drift
rw_sim = 1000
rw_T = 100

#Sumulate rws. Function returns N x T, transpose to get T x N.
rw = sim_AR1(1.0, mu= 0.0, N = rw_sim, T = rw_T, sigma_w = 1.0).T

#Set max # of covariates. For T obs, a reg. w/constant and T-1 covariates should perfec
max_p = rw_T-1

#Create matrix of covariates.
x = np.vstack([np.arange(1,rw_T+1,1)**p for p in range(1,max_p+1)]).T

#Store dist of MSE per number of covars
p_mse_dict = {}
p_sig_dict = {}
p_ar2_dict = {}

#Regress RW on t, then t and t^2, ...., t...t^p
for covar_idx in range(max_p+1):

    #Initialize lists for MSE computed on each RW,  % of covars sig at 5%, Adj. R2
    p_mse = []
    p_sig = []
    p_ar2 = []

    #repeat each simulated RW
    for rw_idx in range(rw_sim):

        model = sm.OLS(endog = rw[:,rw_idx],
                       exog = sm.add_constant(x[:,0:covar_idx])).fit(cov_type="HAC",
                                                                     cov_kwds={"maxlags

        #Compute MSE
        p_mse.append(np.mean((model.resid)**2))

        #Compute % of estimates significant at 5%
        p_sig.append( np.sum(model.pvalues <= 0.05)/len(model.pvalues) )

        #Adj R2
        with warnings.catch_warnings():
            warnings.simplefilter("ignore")
            p_ar2.append(model.rsquared_adj)

    #Update dictionaries
    p_mse_dict[covar_idx] = p_mse
    p_sig_dict[covar_idx] = p_sig
```

```
    p_ar2_dict[covar_idx] = p_ar2

#Check time
t1 = time.time()
print(f"{t1-t0}s")

#Plot results
fig, axs = plt.subplots(nrows=3, ncols=1, figsize=(15,10), sharex=True)
axs[0].plot(p_mse_dict.keys(), [np.mean(v) for v in p_mse_dict.values()], label="MSE",
axs[0].set_ylabel("MSE")
axs[0].set_xlabel("Number of Covariates")
axs[0].set_title("Mean Resid. MSE")
axs[0].grid()

axs[1].plot(p_sig_dict.keys(), [np.mean(v) for v in p_sig_dict.values()], label="% Sig.
axs[1].set_ylabel("% Sig.")
axs[1].set_xlabel("Number of Covariates")
axs[1].set_title("Avg. % of Significant Covariates")
axs[1].grid()

axs[2].plot(p_ar2_dict.keys(), [np.mean(v) for v in p_ar2_dict.values()], label="Adj. R
axs[2].set_ylabel("Adj. R2")
axs[2].set_xlabel("Number of Covariates")
axs[2].set_title("Avg. Adjusted R2")
axs[2].set_xticks(np.arange(0,max_p+1,5))
axs[2].grid()

plt.suptitle(f"In Sample Model Performance per P Covariates, T={rw_T}, n={rw_sim} Rando
plt.tight_layout()
plt.show()
```

```
491.71972846984863s
```



The elbow appears at 5 covariates. Using merely t, t^2, t^3, and t^4 as covariates suffices to model a random walk with 100 observations. Overfitting can occur when the number of covariates is quite small relative to the number of dependent observations, hence the importance of out of sample testing. Average Adjusted R^2 exceeds 70% and at least 3 covariates are significant at a 5% level on average. Using just a time trend, t, produces regression fits with an average R^2 over 40%.

Such regressions highlight the perils of relying on in-sample statistical measures of regression performance in lieu of economic content. It isn't wildly implausible that markets revenue resembles a random walk, or is hard to statistically distinguish from a random walk in a short sample. As I demonstrate below, regressions with stationary variables can still generate spurious results.

1b. Spurious Regression

An AR(1) process is stationary if $|\phi| < 1$ for $X_t = \mu + \phi X_{t-1} + \epsilon_t$. The phenomenon of spurious regression can occur for near random walks, which are AR(1) processes with $\phi \approx 1$. Using the elbow of 5 covariates above, I simulate $N = n + 5$ near unit root AR(1)s:

$$Y_{1,t} = \phi Y_{1,t-1} + e_{1,t}$$
$$\vdots$$
$$Y_{n,t} = \phi Y_{n,t-1} + e_{n,t}$$

and simulate a fixed set of 5 other near unit root AR(1)s.

$$X_{n+1,t} = \phi X_{n+1,t-1} + e_{n+1,t}$$
$$\vdots$$
$$X_{N,t} = \phi X_{N,t-1} + e_{N,t}$$

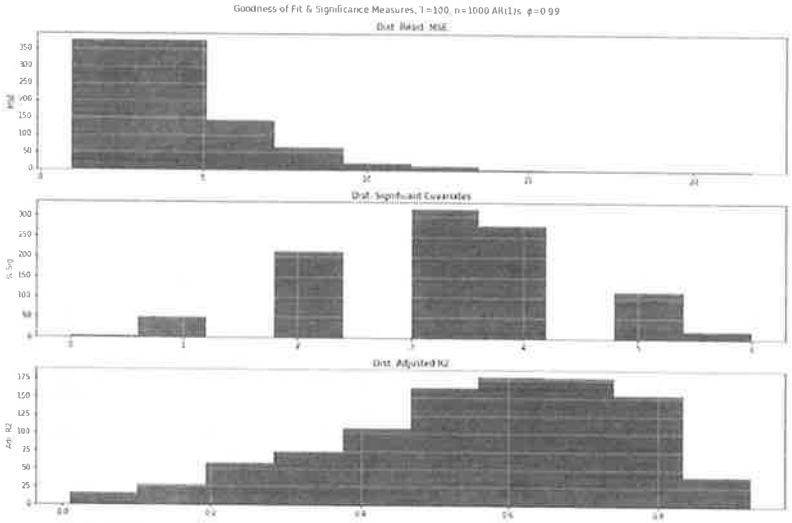
for $\vec{e} \sim N(\vec{0}, I_{N \times N})$. For $i = 1...n$, $j = 1...5$, $j \neq i$, I fit models:

$$Y_{i,t} = \beta_{0,i} + \sum_j \beta_{j,i} X_{n+j,t} + u_{i,t}$$

I plot the distributions of R^2, p-values, and MSE below.

```
#Simulate near RWs. Use phi = 0.99, 20% burn in. 5 near RWs as covariats.
near_rw = []
phi = 0.99
x_len = 5
burn = 0.20

#Construct AR(1)s. Use 0.0 as initial value.
for i in range(rw_sim+x_len):
    near_rw_i = [0.0]
    for t in range(int(rw_T*(1.0 + burn))):
        near_rw_i.append(phi*near_rw_i[-1]+np.random.normal(0,1))
    near_rw.append(near_rw_i[-rw_T:])
```

```
#Check dimensions, should be T X (n+5). Each col is a separate near RW.
near_rw_Y = np.array(near_rw).T[:,:-x_len]
assert(near_rw_Y.shape == (rw_T, rw_sim))

#Set the last cols as the covariates. Fix set of covariates.
near_rw_X = np.array(near_rw).T[:,-x_len:]

#Hold results per number of covars
j_r2, j_p, j_mse = [], [], []

#For each Y, reg on x1...x5
for i in range(rw_sim):

    #Store i'th col as dep var
    Y = near_rw_Y[:,i]

    #Perform regression
    model_Y = sm.OLS(Y, sm.add_constant(near_rw_X)).fit(cov_type="HAC", cov_kwds={"maxl

    #Compute MSE
    j_mse.append(np.mean((model_Y.resid)**2))

    #Compute # of estimates significant at 5%
    j_p.append( np.sum(model_Y.pvalues <= 0.05) )

    #Adj R2
    with warnings.catch_warnings():
        warnings.simplefilter("ignore")
        j_r2.append(model_Y.rsquared_adj)

#Distribution of error measures
fig, axs = plt.subplots(nrows=3, ncols=1, figsize=(15,10), sharex=False)
axs[0].hist(j_mse)
axs[0].set_ylabel("MSE")
axs[0].set_title("Dist. Resid. MSE")
axs[0].grid()

axs[1].hist(j_p)
axs[1].set_ylabel("% Sig.")
axs[1].set_title("Dist. Significant Covariates")
axs[1].grid()

axs[2].hist(j_r2)
axs[2].set_ylabel("Adj. R2")
axs[2].set_title("Dist. Adjusted R2")
axs[2].grid()

plt.suptitle(f"Goodness of Fit & Significance Measures, T={rw_T}, n={rw_sim} AR(1)s: "+
plt.tight_layout()
plt.show()

#Summary stats
print("Summary Statistics")
sr_results = pd.DataFrame({"R2":j_r2,"% P":j_p, "MSE": j_mse})
display(sr_results.describe(percentiles=[0.01,0.25,0.50,0.75,0.95,0.99]).T)

#Count number of regs that satisfy MRM criteria
print("% of regressions with Adj. R2 >= 20% and all variables significant:")
print(len(sr_results[(sr_results.R2 >= 0.20) & (sr_results["% P"]==5)])/len(sr_results)
```



Summary Statistics

	count	mean	std	min	1%	25%	50%	75%	95%	99%
R2	1000.0	0.561613	0.191269	0.009444	0.077948	0.437412	0.584580	0.708990	0.823409	0.882891
% P	1000.0	3.249000	1.137670	0.000000	1.000000	2.000000	3.000000	4.000000	5.000000	6.000000
MSE	1000.0	4.166017	2.364687	0.942536	1.439276	2.540830	3.585943	5.088654	8.522716	12.294078

```
% of regressions with Adj. R2 >= 20% and all variables significant:
0.117
```

Regressing a near random walk on 5 other near random walks can produce R^2 values over 60%, with 4 out of 6 (including the constant) variables significant at 5% using Newey-West standard errors, just by chance. The R^2 distribution does show negative skewness, and the mode appears to be well above the 20% threshold required by MRM. A significant percentage of simulated series satisfy MRM's criteria of 20% adjusted R^2 and all variable significant at 5%.

2. ADF Power for Small T

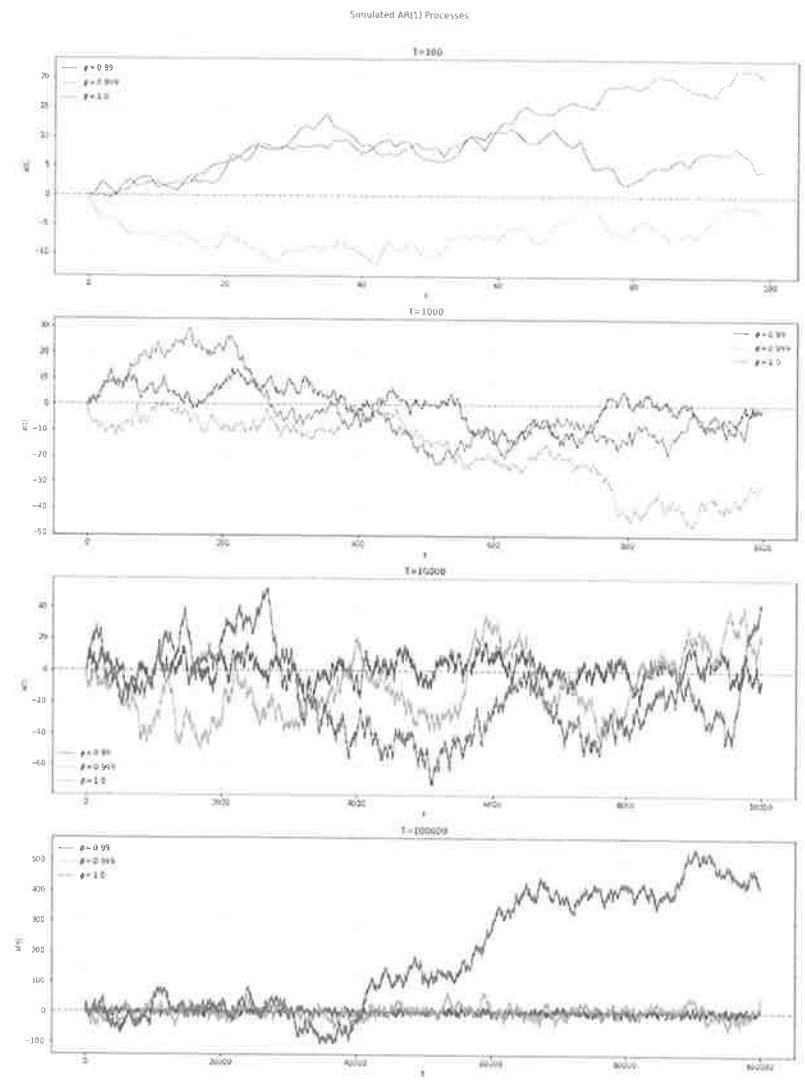
The validation process for Markets PPNR models includes Augmented Dickey-Fuller tests for the stationarity of the dependent and independent variables used in the regression, and the regression residuals. The null hypothesis is that the sample contains a unit root, meaning it is non-stationary. MRM has cited test failures based on p-values very close to the 5% significance level. In this section, I explain why in such cases, using the software output p-values can lead to erroneous conclusions.

First, ADF tests have lower power against near unit-root alternatives. For intuition, consider three AR(1) processes:

$$(1)\ X_t = 0.99X_{t-1} + e_t$$
$$(2)\ X_t = 0.999X_{t-1} + e_t$$
$$(3)\ X_t = X_{t-1} + e_t$$

In a short sample, e.g. T=100, it is difficult to visually distinguish processes (1) and (2), which are stationary, from (3), which is a random walk. At T=1,000, it becomes clearer that process (1) is stationary, because it approaches the long run mean of 0. At T=10,000, the difference between processes (2) and (3) becomes clear, and at T=100,000, it's obvious.

```
#Plot RW and near RWs
fig, axs = plt.subplots(nrows=4, ncols=1, figsize=(15,20))

#Sim RWs at phi=0.99, 0.999,1.0
ar1_99 = sim_AR1(phi=0.99, mu= 0.0, N = 1, T = 100_000, sigma_w = 1.0, burn_in=0.0)[0]
ar1_999 = sim_AR1(phi=0.999, mu= 0.0, N = 1, T = 100_000, sigma_w = 1.0, burn_in=0.0)[0
ar1_1 = sim_AR1(phi=1.0, mu= 0.0, N = 1, T = 100_000, sigma_w = 1.0, burn_in=0.0)[0]

#Subplots for T=100, 1000, 10K, 100K
for idx, max_T in zip([0,1,2,3],[100,1000,10_000,100_000]):
    axs[idx].plot(ar1_99[:max_T], label=r"$\phi=$"+str(0.99))
    axs[idx].plot(ar1_999[:max_T], label=r"$\phi=$"+str(0.999))
    axs[idx].plot(ar1_1[:max_T], label=r"$\phi=$"+str(1.0))
    axs[idx].set_xlabel("t")
    axs[idx].set_ylabel("x(t)")
    axs[idx].axhline(0, linestyle="--")
    axs[idx].grid()
    axs[idx].legend()
    axs[idx].set_title(f"T={max_T}")

#Set title. Adjust y position.
plt.suptitle("Simulated AR(1) Processes", x=0.5, y=1.01)
plt.tight_layout()
plt.show()
```



The result follows from the persistence of shocks in an AR(1). Recursively substitute:

$$
\begin{aligned}
X_t &= \mu + \phi X_{t-1} + \epsilon_t \\
X_t &= \mu + \phi(\mu + \phi X_{t-2} + \epsilon_{t-1}) + \epsilon_t \\
&\vdots \\
X_t &= \mu \sum_{j=0}^{p-1} \phi^j + \phi^p X_{t-p} + \sum_{j=0}^{p-1} \phi^j \epsilon_{t-j}
\end{aligned}
$$

for $|\phi| < 1$, as $p \to \infty$, using the convergence of an infinite geometric series:

$$
\begin{aligned}
X_t &= \frac{\mu}{1-\phi} + \sum_{j=0}^{\infty} \phi^j \epsilon_{t-j} \\
E(X_t) &= \frac{\mu}{1-\phi}
\end{aligned}
$$

and the process tends to revert to its mean. When $\phi = 1.0$, this is not the case. For $\phi = 0.99$, $\phi^{1000} \approx 0$, but for $\phi = 0.999$, $\phi^{1000} = 0.3677$, so the near stationary process still shows meaningful dependence on lagged value of X_{t-1000}. In practice, getting 1000 observations of a time series is rare. Assuming monthly or quarterly data, 120 or 40 observations, respectively is typical.

Even with a large sample size, a limitation of the ADF test for a unit root in the time series sample is that critical values for the test statistic must be derived using simulation, because the test statistic doesn't follow a standard distribution under the null hypothesis. MacKinnon (2010) provides critical values for cointegration and unit root tests based on simulation. He uses response surface regressions to relate critical values to sample size, and therefore estimate CVs for any finite sample using the tables in the Appendix (e.g. N=1, no constant, 5%, T=40):

$$
\begin{aligned}
\text{Est. CV} &= \beta_\infty + \frac{\beta_1}{T} + \frac{\beta_2}{T^2} + \frac{\beta_3}{T^3} \\
&= -1.94100 - \frac{0.2686}{40} - \frac{3.365}{40^2} + \frac{31.223}{40^3}
\end{aligned}
$$

Note, β_∞ is the estimated asymptotic critical value, and β_i is the coefficient on covariate $1/T^i$ for $i = 1, 2, 3$. As $T \to \infty$, the quotient terms drop out.

Statsmodels's implementation of the ADF test provides p-values and critical values using MacKinnon's approximation. The documentation specifically notes that "If the p-value is close to significant, then the critical values should be used to judge whether to reject the null" (Statsmodels 2025).

I demonstrate that even using the prescribed estimated CVs can lead to erroneous conclusions for the small sample case. The procedure is as follows:

1. Simulate N = 10,000 driftless random walks of length T = 40, 100, 250, 500, 750, and 1000. Errors e_t are IID normal, a requirement to use MacKinnon's table values.

$$
\begin{aligned}
X_{i,t} &= X_{i,t-1} + e_t \\
e_t &\overset{IID}{\sim} N(0,1) \\
t &= 1 \ldots T, i = 1 \ldots N
\end{aligned}
$$

1. Compute the Augmented Dickey-Fuller's test statistic, approximate p-value, and estimated critical value on each simulated path using Statsmodel's function, setting the regression type flag to "n", corresponding to the case of no constant and no trend. It is appropriate to impose this restriction on the specification because I defined the process ex-ante. With real data, a more defensible choice would be "c", corresponding to a constant term. Setting the wrong specification for the test can seriously distort the test's size and power. For p lags, the "n" and "c" cases are, respectively:

$$
\begin{aligned}
\text{"n"} &: \Delta X_{i,t} = \gamma X_{i,t-1} + \delta_1 \Delta X_{i,t-1} + \ldots + \Delta_p X_{i,t-p} + e_t \\
\text{"c"} &: \Delta X_{i,t} = \beta_0 + \gamma X_{i,t-1} + \delta_1 \Delta X_{i,t-1} + \ldots + \Delta_p X_{i,t-p} + e_t
\end{aligned}
$$

1. Perform the test using SM's approximate p-value and critical value. Both are based upon MacKinnon 2010. For each i, t, mark true if SM's p-value is less than 5% and separately mark true if the test statistic is less than or equal to the critical value. True means reject the null hypothesis of a unit root present.

1. Per t $\in \{40, 100, 250, 500, 750, 1000\}$, compute the average rate of rejection by summing indicators, where 1 corresponds to True and 0 to False, and dividing by N. Because I specified the data generating process beforehand as a random walk, True corresponds to rejecting a true null hypothesis, and therefore the average is an estimate of the test's Type 1 error rate, which should be capped at 5%.

1. Compare estimated Type 1 error rates, $\hat{\alpha}$, computed using the p-value and critical value, per t, to the 5% significance level. Further compare the distribution of test statistics per t to manually computed critical values, using MacKinnon's paper, and the 5th percentile of the test statistics.

```
#Create a grid for phi and T.
phi_grid = np.array([1.0])
T_grid = [40,100,250,500,750,1000]
adf_grid = pd.DataFrame(data=list(product(phi_grid,T_grid)), columns=["Phi","T"])
adf_grid = adf_grid.sort_values(by="T")

#Initialize empty lists to hold rejection rates and CV
r_H0_rate = [] #reject using p-value
r_H0_rate_cv = [] #reject using CV
cv_used, test_stats, nobs = [], [], []

#Drift or no drift
sim_drift = 0.0

#If no drift, set reg flag to "n"
if sim_drift == 0.0:
    reg_flag = "n"
#Otherwise, set flag to 'c' for constant (the default)
else:
    reg_flag = "c"


#Iterate through the DF rows, extracting phi and T combinations
ctr = 1
for row in adf_grid.iterrows():
```

```
    #Start timing
    t0 = time.time()

    #Simulate 10K AR(1)s with Phi and T.
    test_sim = sim_AR1(row[1]["Phi"], T=int(row[1]["T"]), mu=sim_drift, N=10_000, sigma

    """
    Perform ADF test on each sim AR(1).
    Use default maxlag, 12*(T/100)^(1/4). Constant in reg: X(t) = mu + Phi*X(t-1) + e(t
    For lag order, test down using t-stat.
    """
    reject_H0, reject_H0_cv = [], []
    cvs, stat, n_sm = [], [], []

    for n in range(test_sim.shape[0]):

        #Perform test, compute p-value and test stat
        adf_result = adf(x = test_sim[n,1:], maxlag=None, regression = reg_flag, autola
        stat_adf, p_adf = adf_result[0], adf_result[1]

        #Test at 5% level. If p < 0.05, reject H0 and append True
        reject_H0.append(p_adf < 0.05)

        #Store CV used as well
        cv_adf = adf_result[4]['5%']
        cvs.append(cv_adf)

        #Test at 5% level using CV
        reject_H0_cv.append(stat_adf <= cv_adf)

        #Store test statistic and nobs used for CV
        stat.append(stat_adf)
        n_sm.append(adf_result[3])

    #Compute empirical rejection rate using approx p val and CV
    r_H0_rate.append(sum(reject_H0)/len(reject_H0))
    r_H0_rate_cv.append(sum(reject_H0_cv)/len(reject_H0_cv))
    cv_used.append(cvs)
    test_stats.append(stat)
    nobs.append(n_sm)

    #Track progress
    print(f"{ctr} out of {len(adf_grid)}, {np.round(time.time() - t0,2)}s")

    #Increment ctr
    ctr += 1

#Update DF with rejection rates as new columns
adf_grid["Reject H0"] = r_H0_rate
adf_grid["Reject H0 CV"] = r_H0_rate_cv

#CVs, and test stats as new columns
adf_grid["SM's CVs"] = cv_used
adf_grid["Test Statistics"] = test_stats
adf_grid["NObs Used"] = nobs

#Check results
display(adf_grid)
```

```
#WTF
adf_grid.to_csv("C:/Users/RP64662/Downloads/ADF_Power_Test.csv")
```

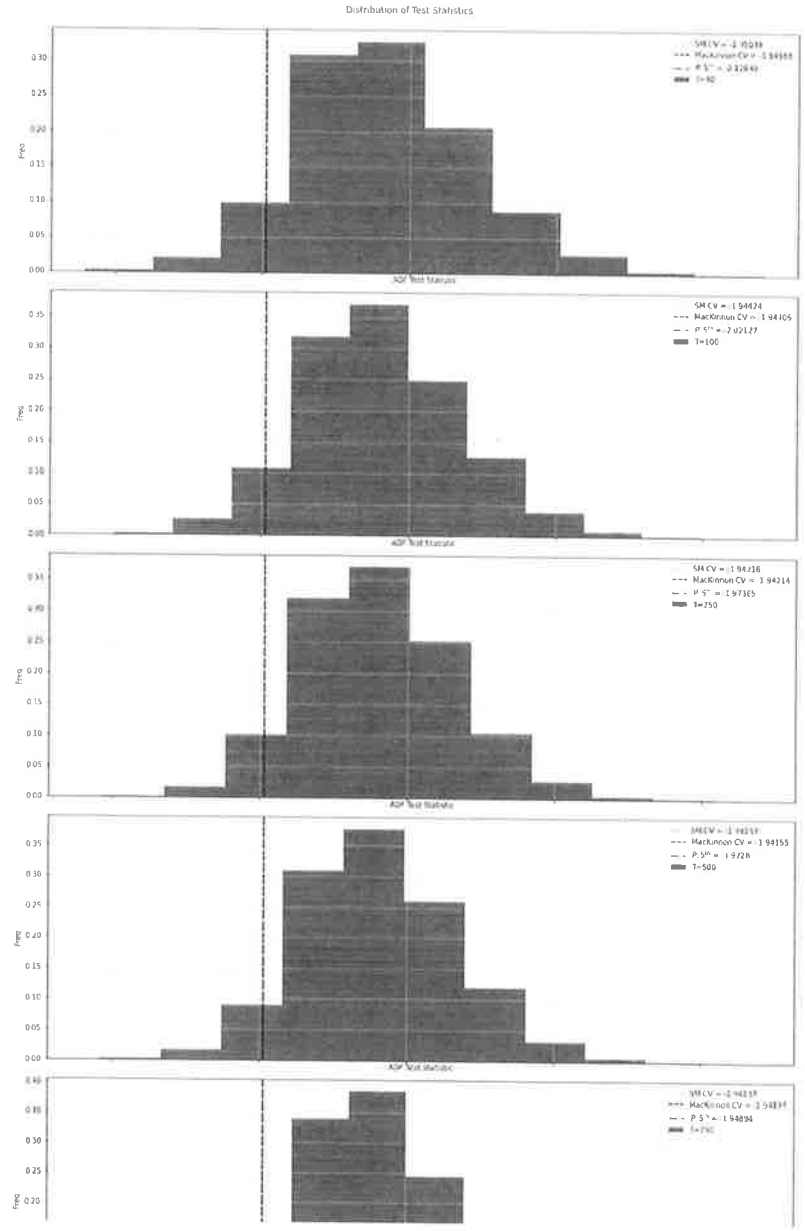
```
1 out of 6, 40.8s
2 out of 6, 54.88s
3 out of 6, 101.72s
4 out of 6, 140.14s
5 out of 6, 586.84s
6 out of 6, 1001.86s
```

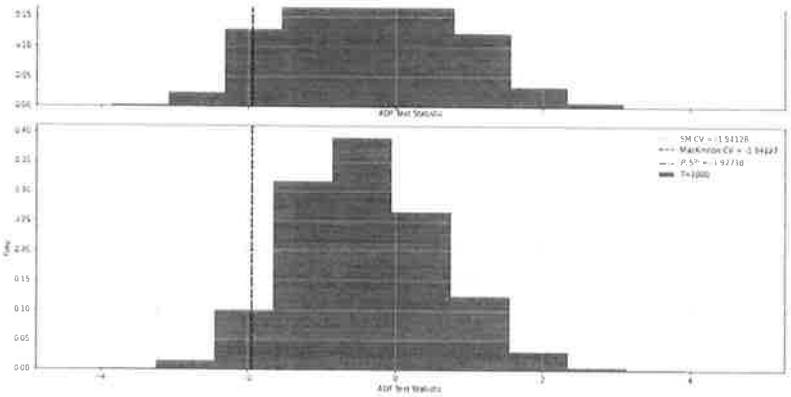
	Phi	T	Reject H0	Reject H0 CV	SM's CVs	Test Statistics	NObs Used
0	1.0	40	0.0694	0.0679	[-1.951016502137187, -1.9498297383000438, -1.9...	[1.7138989671083704, 0.030227877673721655, -1...	[34, 38, 33, 32, 36, 38, 28, 38, 28, 32, 28, 2...
1	1.0	100	0.0596	0.0593	[-1.9445291430314313, -1.9440580172802149, -1...	[-0.9131182167872453, -0.18086923842042882, -1...	[86, 98, 98, 94, 98, 92, 88, 87, 98, 87, 95, 9...
2	1.0	250	0.0535	0.0535	[-1.9421357293965669, -1.9422015054082429, -1...	[-2.5158132209951596, 1.5655766332833956, -0.0...	[248, 235, 248, 239, 242, 248, 243, 234, 247, ...
3	1.0	500	0.0539	0.0539	[-1.9415726837596574, -1.9415526729537138, -1...	[0.16096422946504144, 0.01356125510012628, 0.2...	[481, 498, 480, 480, 481, 489, 486, 488, 491, ...
4	1.0	750	0.0506	0.0505	[-1.9413731403326997, -1.9413721069683731, -1...	[-0.8797067561876631, 0.09589683614715606, -0...	[732, 734, 748, 734, 729, 728, 740, 746, 733, ...
5	1.0	1000	0.0486	0.0484	[-1.9412769799410616, -1.9412766946917843, -1...	[-0.7866917961065525, -0.24256938319164717, -1...	[982, 983, 976, 982, 978, 981, 998, 998, 985, ...

Examine distribution of test statistics per T.

```
#Set subplots, one per unique T in the grid
fig, axs = plt.subplots(nrows = len(T_grid), ncols=1, figsize=( 15 , 5*len(T_grid)), sq

#Per T, plot distribution of test statistics
for ctr, t in enumerate(T_grid):

    #Histogram of test statistics
    axs[ctr][0].hist(adf_grid[adf_grid["T"]==t]["Test Statistics"], density=True, label
    axs[ctr][0].set_ylabel("Freq.")
    axs[ctr][0].set_xlabel("ADF Test Statistic")

    #Verticle lines at SM's CV
    med_cv = np.median(adf_grid[adf_grid["T"]==t]["SM's CVs"].values[0])
    axs[ctr][0].axvline(x=med_cv, linestyle="--", color="blue", label=f"SM CV = {np.rou

    #And MacKinnon's CV based on formula. Use mode of SM's num of obs used across sims
    modal_T = sp.stats.mode(adf_grid[adf_grid["T"]==t]["NObs Used"].values[0], keepdims
```

```
    mk_cv = adf_cv(T=modal_T)
    axs[ctr][0].axvline(x=mk_cv, linestyle="--", color="black", label=f"MacKinnon CV =

    #Finally, percentile of test stats (CV s.t. reject 5%)
    p_5 = np.percentile(adf_grid[adf_grid["T"]==t]["Test Statistics"].values[0], 5)
    axs[ctr][0].axvline(x=mk_cv, linestyle="dashdot", color="blue", label=r'$P.5^{th}$

    #Legend and grid
    axs[ctr][0].grid()
    axs[ctr][0].legend()

#Set title and display
plt.tight_layout()
plt.suptitle("Distribution of Test Statistics", y=1.01)
plt.show()
```



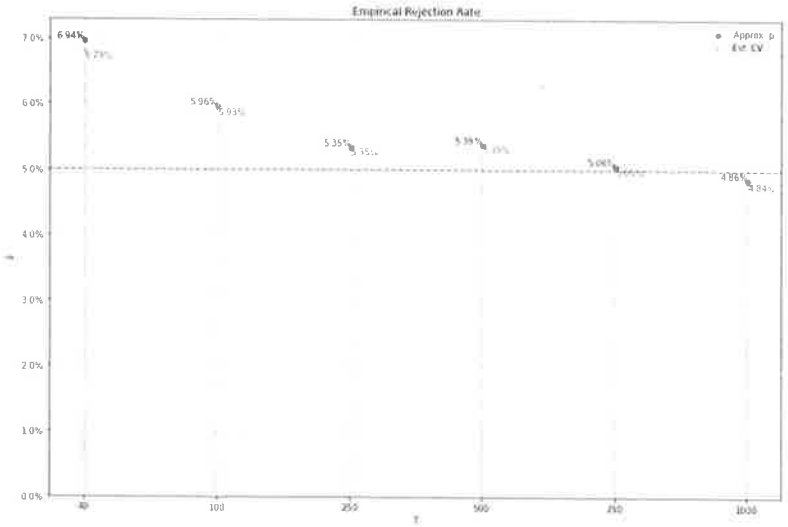


As the number of lags, and therefore number of observations used, varies per simulation, I report the median critical value used by SM and compare to MacKinnon's formula output using the median number of observations for T. Statsmodel's critical values and MacKinnon's critical values are quite close, with the discrepancy shrinking as T increases to 1000. This is to be expected, because SM uses MacKinnon's approximation.

For T=40, the 5th percentile of the test statistics is about -2.12, which is greater (in magnitude) than the corresponding CV of -1.94, indicating that for small T, the estimated critical value isn't conservative enough. The rejection region implied by the estimated CV at 5% and T=40 is, per the simulation results, too wide, and therefore I expect the actual Type 1 error rate, $\hat{\alpha}$, will exceed the 5% level. At T=250 and above, SM's CV and MacKinnon's CV are closely aligned.

Below I plot empirical rejection rates using approximate p-values and estimated critical values to reject the null hypothesis.

```
#Bar Plot
plt.figure(figsize=(15,10))
plt.bar(adf_grid["T"].astype(str), 100.0*adf_grid["Reject H0"], width=0.02, alpha=0.10,

#Add dots at alpha hat. First set is using approx p, second using CV.
plt.scatter(adf_grid["T"].astype(str), 100.0*adf_grid["Reject H0"], marker="o", label="
plt.scatter(adf_grid["T"].astype(str), 100.0*adf_grid["Reject H0 CV"], marker="x", labe

#Format y axis in %
plt.gca().yaxis.set_major_formatter(mtick.PercentFormatter())

#Horizontal line at 5% level
plt.axhline(y=5, linestyle="--")

#Annotate
for i in range(len(adf_grid)):

    #p
    x_coord = adf_grid.iloc[i]["T"].astype(str)
    y_coord = 100.0*adf_grid.iloc[i]["Reject H0"]
    s_txt = f"{np.round(y_coord,2)}%"
    plt.text(x = x_coord, y= y_coord, s=s_txt, ha='right', va='bottom',color="blue")

    #cv
    y_coord = 100.0*adf_grid.iloc[i]["Reject H0 CV"]
    s_txt = f"{np.round(y_coord,2)}%"
    plt.text(x = x_coord, y= y_coord, s=s_txt, ha='left', va='top', color="red")

#Set labels and display
plt.grid()
plt.xlabel("T")
plt.ylabel(r"$\hat{a}$")
plt.title("Empirical Rejection Rate")
plt.legend()
plt.show()
```



Indeed, the empirical Type 1 error rate is nearly 7%, which is noticeably higher than the test level of 5% for T=40. Error rates are much closer to 5% for T>100, with the empirical rate consistently a few basis points closer to 5% using estimated critical values instead of approximate p-values. Using estimated critical values is superior, but doesn't suffice for the T=40 or T=100 case.

Markets PPNR models typically use 10 years of quarterly data, owing to data availability and quality concerns, changes in the business's strategy, and to exclude any structural breaks associated with the GFC. In the small sample case, it is therefore entirely possible to draw the wrong conclusion about the stationarity of a time series of revenue actuals using an ADF test at 5%.

3. Multiple Testing

The model development and validation process is intended to exclude models based on spurious regressions. Validators examine the results of statistical tests for the stationarity of the transformed dependent variable, transformed independent variable, and model residuals, serial correlation in the model's residuals, the model's adjusted R^2 (which must exceed 20%), and p-values computed using heteroskedasticity and autocorrelation robust standard errors. Furthermore, the variable selection process typically involves computing the correlation between the dependent variable and all candidate macroeconomic covariates. Ideally, model developers use economic rationale to form a directional hypothesis about the relationship between revenue and MEVs prior to correlation analysis. Developers generally test regression submodels built using subsets of candidate covariates, assessing how model fit and parameter estimates change with the incremental addition of covariates.

Statistical tests for Markets PPNR models typically use a 5% significance level. To assess the aggregate stringency of these tests, I simulate N=1,000 quarterly revenue actuals in levels, effectively using normal draws for the logarithmic change in revenue such that simulated revenue is non-negative, and then first-difference. I then perform multivariate correlation analysis using the universe of EFT provided first-differenced US macroeconomic variables, spanning 2009 Q2 - 2025 Q1 (T=64) inclusive. MEVs include credit spreads, CDS spreads, US Treasury yields, swap rates, equity indices and EPS, currency and commodity volatility, and equity and fixed income implied volatility. For each simulated path for the change in revenue, I select the 3 first-differenced MEVs with the highest absolute correlations, and fit regressions on all 7 submodels: 3 univariate models, 3 bivariate models, and 1 trivariate model. All regressions include a constant, and use Newey-West standard errors. For each regression, I retain the adjusted R^2, the p-values, ADF stationarity test results for the dependent and independent variables, and finally I test the residuals for stationarity and serial correlation.

After collecting results, I apply the stated criteria for model development and validation and count how many regression models satisfy all criteria, or at least one criterion, for different combinations of criteria. One criterion is least stringent, all criteria is most stringent.

```
#Pull MEV actuals
try:
    mev_path = pd.read_csv("C:/Users/RP64662/Downloads/Mar2025_Q_MEV_Dump.csv")
    mev_path = mev_path.set_index("eom")
except FileNotFoundError:
    #Pull MEVs for 2Q QMMF
    path_query = """Select * From Vw_Ccar_Interpolated
                    Where  Version='v_Mar25'
                    And Scenario In ('Actuals')
                 """
    mev_path = pd.DataFrame(pd.read_sql_query(path_query, con=init_sql_connection()))

    #Convert EOM to date time
    mev_path["eom"] = pd.to_datetime(mev_path["eom"])

    #Pivot on sas variable and scenario
    mev_path = mev_path.pivot(index="eom", columns=["sas_variable"], values="amount")

    #Quarterly, use EOP
    mev_path = mev_path.resample("Q").last()

    #Dump to CSV
    mev_path.to_csv("C:/Users/RP64662/Downloads/Mar2025_Q_MEV_Dump.csv")

#Set MEVs to use. US only, no EOP or LVL suffix to avoid duplicates.
mev_names = [col for col in mev_path.columns if "US_" in col and "EOP" not in col and "
mevs = mev_path[mev_names].diff(1).dropna()

#Simulate revenue
n = 1_000
T = len(mevs)

#Params. Small drift and diffusion.
mu_hat = 0.01
sigma_sq_hat = 0.01

#GBM simulations.
dt = 1.0
drift = (mu_hat - 0.5 * sigma_sq_hat)*dt
diffusion = np.sqrt(sigma_sq_hat)*np.sqrt(dt)

#Brownian motion increments are N(0, sqrt(dt))
bt_shocks = np.random.randn(n, T)
ln_ret = drift + diffusion*bt_shocks
cum_ln_ret = np.cumsum(ln_ret, axis=1)

#Exponentiate cum log returns, mutply by initial rev. Add R0 as first col.
R0 = 1e8
rev_paths = np.concatenate( (np.full((n,1),R0) ,R0*np.exp(cum_ln_ret)  ), axis=1)

#Transpose, first difference, set index for merge
rev_paths = pd.DataFrame(rev_paths).T.diff(1).dropna()
rev_paths.columns = [f"Rev{i}" for i in rev_paths.columns]
rev_paths.index = mevs.index

#Append to MEVs
mevs = pd.concat([mevs, rev_paths], axis=1)
mevs = mevs.dropna()

#Compute correlation
corr_df = mevs.corr()
corr_df = corr_df[rev_paths.columns]
corr_df = corr_df[~corr_df.index.isin(rev_paths.columns)]
corr_df = np.abs(corr_df)

#For each revenue path, pick out largest 3 variables by abs correlation
reg_results = []
for path in rev_paths.columns:

    #Set Dep var
    dep_var = path

    #Set covariates using correlation matrix top 3 results
    indep_vars = list(corr_df[path].sort_values(ascending=False).head(3).index)

    #Test stationarity of dep var and indep vars
    y_stat = adf(x = mevs[dep_var], maxlag=None, regression = "c", autolag="t-stat")[1]
    x_stat = []
    for iv in indep_vars:
```

```
        x_stat.append(adf(x = mevs[iv], maxlag=None, regression = "c", autolag="t-stat"

    #Full sample regression. Test on subregression (7: 3c1 univariate, 3c2 bivariate, 1
    sub_models = [list(it) for it in list(chain.from_iterable(combinations(indep_vars,

    #Run 7*1K regressions, for 7 submodels associated w/each simulated dep var path
    for sub_cov in sub_models:

        #Set dependent variable and independent variable, include constant
        Y = mevs[dep_var]
        X = sm.add_constant(mevs[sub_cov])

        #Fit regression using Newey-West HAC robust SE
        model = sm.OLS(Y, X).fit(cov_type="HAC", cov_kwds={"maxlags":int(len(mevs)**0.2

        #Test residuals for stationarity. No constant or trend. Return true if stationa
        resid_stat = adf(x = model.resid, maxlag=None, regression = "n", autolag="t-sta

        #Test residuals for serial correlation at 4 lags (Q=4:1 year)
        ac_acf_p = acorr_ljungbox(x = model.resid, lags=[4], model_df = int(model.df_mo

        #H0: no serial correlation, return true if serial correlation present.
        resid_ac =  ac_acf_p < 0.05

        #Append results as dictionary
        reg_results.append({"Dependent Variable":dep_var,
                            "Independent Variables":sub_cov,
                            "Coefficients":model.params.values,
                            "NW P-Values":model.pvalues.values,
                            "Adj. R2":model.rsquared_adj,
                            "Y Stat.": y_stat,
                            "X Stat.": x_stat,
                            "Resid. Stat.":resid_stat,
                            "Resid. Auto Corr.":resid_ac})

#Store results as df
reg_results = pd.DataFrame(reg_results)

#Set conditions to filter results. First, on p-values, is all (inc. intercept) sig at 5
pval_cond1 = reg_results["NW P-Values"].apply(lambda x: np.all(x[0:]<=0.05))

#Second, on p-values, is all (excl. intercept) sig at 5%:
pval_cond2 = reg_results["NW P-Values"].apply(lambda x: np.all(x[1:]<=0.05))

#Third is Adj. R2 over 20%
ar2_cond = reg_results["Adj. R2"].apply(lambda x: x >= 0.20)

#Fourth is dep and indep variables stationary
stat_cond = (reg_results["Y Stat."] == True) & (reg_results["X Stat."].apply(lambda x:

#Fifth is stationarity of residuals
resid_stat_cond = (reg_results["Resid. Stat."] == True)

#6th is residuals show no serial correlation. Flag is serial correlation present (p < 0
resid_ac_cond = (reg_results["Resid. Auto Corr."] == False)

#Use conditions to identify spurious reg. Progressively increase number of requirements
#p < 5% (no intercept), R2 > 20%, Y and X stationary
print("Requiring all p-values < 5%, excl. intercept, Adj. R2 > 20%, and all variables s
      f"{len(reg_results[(pval_cond2) & (ar2_cond) & stat_cond])}/{len(reg_results)} re
```

```
display(reg_results[(pval_cond2) & (ar2_cond) & stat_cond].sort_values(by="Adj. R2", as

#p < 5% (inc intercept), R2 > 20%, Y, X, resid stationary, resid serially uncorrelated
print("Requiring all p-values < 5%, including intercept, Adj. R2 > 20%, all variables &
      f"{len(reg_results[(pval_cond1) & (ar2_cond) & stat_cond & resid_stat_cond & resi
display(reg_results[(pval_cond1) & (ar2_cond) & stat_cond & resid_stat_cond & resid_ac_
```

```
Requiring all p-values < 5%, excl. intercept, Adj. R2 > 20%, and all variables stationar
y: 135/7000 regressions spurious.
```

	Dependent Variable	Independent Variables	Coefficients	NW P-Values	Adj. R2	Y Stat.	X Stat.
1585	Rev226	[US_EQ_INDU, US_E_CDSR]	[249369.01546856857, 3517.6827739348205, 28566...	[0.8587549667679475, 1.4496089703891072e-05, 3...	0.415916	True	[True, True, True]
1392	Rev198	[US_E_EXPORTS_G, US_PR_GAS, US_E_EMPL]	[1719490.8123885242, -546.2496008192079, -6100...	[0.1619398711372414, 0.00790887812598365, 1.11...	0.409557	True	[True, True, True]
1389	Rev198	[US_E_EXPORTS_G, US_PR_GAS]	[1632918.4192567095, -880.4041155438797, -5590...	[0.2079796684177413, 1.121634607725051e-06, 1...	0.371969	True	[True, True, True]
1391	Rev198	[US_PR_GAS, US_E_EMPL]	[1217744.329261582, -6814619.287821911, -2931...	[0.3259386294349731, 1.9979697201815882e-05, 7...	0.370454	True	[True, True, True]
1587	Rev226	[US_E_CDSR, US_EQ_MSEUACWF]	[738242.1999046595, 29677808.399087317, 145794...	[0.589970942875454, 1.5009086733180597e-09, 1...	0.364829	True	[True, True, True]

```
Requiring all p-values < 5%, including intercept, Adj. R2 > 20%, all variables & resid s
tationary, and resid serially uncorrelated: 19/7000 regressions spurious.
```

	Dependent Variable	Independent Variables	Coefficients	NW P-Values	Adj. R2	Y Stat.	Sta
1792	Rev256	[US_IR_3M_FCP_AA]	[4272887.618805755, -20275192.651307054]	[0.004051833317793809, 2.3975552761505225e-16]	0.309747	True	[Tru Tru Tru
5884	Rev840	[US_E_RSEAS, US_IR_1Y_RFRSWP]	[13782096.587293904, -41985.61312165155, -2919...	[0.0024189331339248554, 6.213489979097034e-05,...	0.307733	True	[Tru Tru Tru
5883	Rev840	[US_E_RSEAS, US_IR_2Y_RFRSWP]	[13407233.001470491, -40496.95479466915, -2689...	[0.0034630733170443453, 4.778266175014729e-05,...	0.301002	True	[Tru Tru Tru
5729	Rev818	[US_CR_HY_CCC, US_EQ_C_CDS5Y]	[-1747638.009837024, -897765.5316409294, -4166...	[0.018807808736013724, 0.001359068627069614, 0...	0.291823	True	[Tru Tru Tru
6114	Rev873	[US_IR_3M_FCP_AA, US_EQ_INDU]	[10126588.840906445, 29163201.35528246, -5985...	[0.011570685882739122, 5.7837439479965646e-05,...	0.271506	True	[Tru Tru Tru

Absent context, some regressions are plausible based on the combinations of MEVs selected. Markets PPNR models have used just one variable, and often include just two economic variables.

With all criteria, the probability of a spurious regression passing all tests is about 0.3%. The requirements for the stationarity and absence of serial correlation in the residuals don't meaningfully affect this result. However, not requiring the intercept to be significant at 5% raises the probability to about 2%. This may be an artifact of the data generating process for the simulated revenue paths, which uses a near zero drift. Accordingly, the interpretation of the intercept in the first-differenced regression model as the average change in revenue conditional on covariate values of zero suggests the estimated intercept should be statistically indistinguishable from zero.

Each requirement represented as a boolean filter in the code above can be thought of as a separate test, with a null hypothesis of no effect, or equivalently, the regression is spurious. The R^2 requirement, for example, is the minimum explanatory power for the model. If it exceeds 20%, reject the null and conclude the relationship is non-spurious, and if it is below, conclude the model has insufficient explanatory power and fail to reject the null, concluding the regression is spurious. The p-value requirement states all variables must be significant at 5% using Newey-West standard errors. If so, reject the null. If not, fail to reject the null because at least one parameter estimate is statisticaly zero, and conclude that the regression is spurious.

To assess the marginal effect of each additional requirement in terms of pruning spurious regressions, I examine every combination of the following requirements:

- All p-values (including the intercept) are less than 5%
- Adjusted R^2 exceeds 20%
- Dependent and independent variables are stationary
- Residuals are stationary
- Residuals do not exhibit serial correlation

and count how many spurious regressions satisfy at least one requirement, and how many spurious regressions satisfy all requirements in the combination.

```
#Use dictionary to identify requirements (reqs are boolean column masks)
cond = {"P-Values":(pval_cond1),
        "Adj. R2":(ar2_cond),
        "Stat.":stat_cond,
        "Resid. Stat":resid_stat_cond,
        "Resid AC":resid_ac_cond}

#Obtain all combinations of requirements.
req_combos = [list(it) for it in list(chain.from_iterable(combinations(cond.keys(), r)

#Identify the indices associated with at least one of the reqs, and associated with all
combo_df = []
for sub_reqs in req_combos:

    #Empty lists for or, and
    or_idxs = []
    and_idxs = []

    #Iterate through individual requirements
    for req in sub_reqs:

        #Update or (indices that satisfy one req or the others)
        or_idxs += list(reg_results[cond[req]].index)

        #Update and (indices satisfying all req)
        and_idxs.append(list(reg_results[cond[req]].index))


    #Remove duplicates in list of indices for or
    or_idxs = list(set(or_idxs))

    #Find intersection of sublists in and_idxs
    intersection_set = set(and_idxs[0])
    for i in range(1, len(and_idxs)):
        current = set(and_idxs[i])
        intersection_set = intersection_set.intersection(current)
    and_idxs = list(intersection_set)

    #Store results as DF
    combo_df.append({"Reqs":sub_reqs,"Or Count":len(or_idxs),"And Count": len(and_idxs)

#Display and add %
combo_df = pd.DataFrame(combo_df)
combo_df["Or %"] = combo_df["Or Count"].apply(lambda x: f"{np.round(100.0*x/len(reg_res
combo_df["And %"] = combo_df["And Count"].apply(lambda x: f"{np.round(100.0*x/len(reg_r

display(combo_df.sort_values(by=["And Count"]))
```

	Reqs	Or Count	And Count	Or %	And %
30	[P-Values, Adj. R2, Stat., Resid. Stat, Resid AC]	6882	19	98.31%	0.27%
26	[P-Values, Adj. R2, Stat., Resid AC]	6147	20	87.81%	0.29%
25	[P-Values, Adj. R2, Stat., Resid. Stat]	6582	28	94.03%	0.4%
15	[P-Values, Adj. R2, Stat.]	3848	29	54.97%	0.41%
27	[P-Values, Adj. R2, Resid. Stat, Resid AC]	6864	61	98.06%	0.87%
17	[P-Values, Adj. R2, Resid AC]	5431	66	77.59%	0.94%
16	[P-Values, Adj. R2, Resid. Stat]	6461	87	92.3%	1.24%
5	[P-Values, Adj. R2]	974	94	13.91%	1.34%
28	[P-Values, Stat., Resid. Stat, Resid AC]	6868	132	98.11%	1.89%
19	[P-Values, Stat., Resid AC]	6040	145	86.29%	2.07%
29	[Adj. R2, Stat., Resid. Stat, Resid AC]	6864	147	98.06%	2.1%
22	[Adj. R2, Stat., Resid AC]	6069	162	86.7%	2.31%
18	[P-Values, Stat., Resid. Stat]	6553	178	93.61%	2.54%
6	[P-Values, Stat.]	3617	193	51.67%	2.76%

	Reqs	Or Count	And Count	Or %	And %
21	[Adj. R2, Stat., Resid. Stat]	6514	209	93.06%	2.99%
9	[Adj. R2, Stat.]	3558	224	50.83%	3.2%
23	[Adj. R2, Resid. Stat, Resid AC]	6844	298	97.77%	4.26%
11	[Adj. R2, Resid AC]	5314	332	75.91%	4.74%
20	[P-Values, Resid. Stat, Resid AC]	6850	336	97.86%	4.8%
8	[P-Values, Resid AC]	5271	403	75.3%	5.76%
7	[P-Values, Resid. Stat]	6418	459	91.69%	6.56%
10	[Adj. R2, Resid. Stat]	6379	470	91.13%	6.71%
1	[Adj. R2]	520	520	7.43%	7.43%
0	[P-Values]	548	548	7.83%	7.83%
24	[Stat., Resid. Stat, Resid AC]	6848	2315	97.83%	33.07%
13	[Stat., Resid AC]	5943	2445	84.9%	34.93%
12	[Stat., Resid. Stat]	6479	3112	92.56%	44.46%
2	[Stat.]	3262	3262	46.6%	46.6%
14	[Resid. Stat, Resid AC]	6828	4627	97.54%	66.1%
4	[Resid AC]	5126	5126	73.23%	73.23%
3	[Resid. Stat]	6329	6329	90.41%	90.41%

The 'And %' column shows the percentage of spurious regressions that pass all the tests listed in 'Reqs'. The 'Or %' column shows the percentage of spurious regressions that pass at least one of the tests listed in 'Reqs'. For example, about 90% of spurious regressions pass the stationary residuals test, so the test individually does a poor job of discriminating between valid (non-spurious) and invalid (spurious) regression models.

Examine the results and note the following. First, using just the requirements of p-values less than 5% and adjusted R^2 over 20% suffices to exclude over 99.5% of spurious regressions. Second, using just the p-values requirement, only 7% of spurious regressions pass. This result is consistent with the ADF test power analysis at T=40 conducted in part 2, as the sample size for simulated revenue, at 64, is similar. Third, adding the requirements of stationary and white noise residuals to the p-value, stationary variables, and adjusted R^2 requirements lowers the rate at which spurious regressions are falsely identified as valid by about 0.14%. For white noise residuals specifically, the marginal effect is 0.13%. When the requirements already include stationary independent and dependent variables, the marginal effect of requiring stationary residuals is 0.01%. The residuals tests are largely superfluous.

By the design of the simulation exercise, the null hypothesis of a spurious regression is true. I used a stochastic process to generate the dependent variable, it has no relationship with the macroeconomic covariates. Extending the joint hypothesis framing, the percentage of simulated regression models that pass at least one of the tests (equivalently, at least one Type 1 error, because passing means concluding that the regression is non-spurious, but the null of spurious is true and therefore passing entails rejecting a true H_0) in the set listed in 'Reqs' is the family-wise error rate for that set of requirements. For example, the FWER for the requirements P-Values and Adj. R^2 is about 14%, compared to individual error rates for each requirement of about 7%-8%. Typically, the FWER is higher than the level of each individual test (with more tests, the overall error rate increases significantly), so this is to be expected. The FWER for the requirements P-Values and Adj. R^2 relative to individual error rates indicates a good balance of size and power.

Consequently, it need not be the case that a valid model passes every single requirement, especially the requirements on the model residuals. If a model passes the requirement that all p-values are less than 5%, including that for the estimated intercept, and passes the requirement of adjusted R^2 over 20%, developers and model validators can safely conclude the regression is non-spurious, regardless of properties of the residuals.

Misc: Revenue Simulation

I outline a time series process for simulating revenue. The process incorporates trend, seasonal, cyclical, and irregular components, mimicking the components of Markets PPNR actuals.

$$y_t = \mu_t + \gamma_t + \psi_t + \epsilon_t$$

Unless otherwise specified, intial values are unknown coefficients.

Trend Component μ_t

μ_t is a random walk with stationary stochastic drift:

$$\begin{aligned} \mu_{t+1} &= \mu_t + \beta_t + \eta_t, \eta_t \overset{\text{iid}}{\sim} N(0, \sigma_\eta^2) \\ \beta_{t+1} &= \phi_\beta \beta_t + \zeta_t, \zeta_t \overset{\text{iid}}{\sim} N(0, \sigma_\zeta^2) \\ \beta_1 &\sim N\left(0, \sigma_\zeta^2/(1-\phi_\beta^2)\right) \end{aligned}$$

Seasonal Component γ_t

Use time-varying dummies, where the seasonal pattern evolves slowly over time:

$$\begin{aligned} \gamma_{t+1} + \gamma_t + \ldots + \gamma_{t-S+2} &= \omega_t, \omega_t \overset{\text{iid}}{\sim} N(0, \sigma_\omega^2) \\ \gamma_{t+j} &= \gamma_{t+j-s} \end{aligned}$$

for seasonal length S, e.g. S=4 for quarterly, S=12 for monthly. Equation 1 constraints the the seasonal effects to sum, on average, to 0.

Cycle Component ψ_t

The cyclical component is a stationary ARMA process. It suffices to pick a low order process.

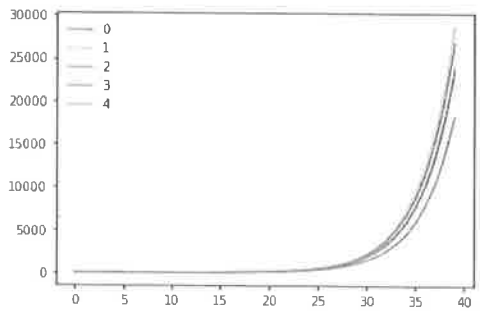
$$\phi_\psi(L)\psi_t = \theta_\psi(L)\xi_t, \xi_t \overset{\text{iid}}{\sim} N(0, \sigma_\xi^2)$$

Irregular Component ϵ_t.

$$\epsilon_t \overset{\text{iid}}{\sim} N(0, \sigma_\epsilon^2)$$

All disturbances are mutually and serially independent. See below for simulation code.

```
#Simulate structural time series model
def sim_ss(N=1000, T=10, S=4,burn_in = 0.20,
           phi_beta = 0.01,  ar_params=[0.5], ma_params=[],
           sigma_eps=None, sigma_eta=None, sigma_zeta=None, sigma_omega=None, sigma_xi=
           S_init = None):
    '''
    Simulate structural time series model (Harvey 1989).
    Args:
        N (int): number of simulations, default 1,000
        T (int): length of simulated time series (in years), default 10
        S (int): frequency of simulated series, e.g. quarterly:4, monthly=12. default 4
        burn_in (float): percent of initial obs to discard, default 20%
        beta_phi (float): AR(1) parameter for stochastic drift, default 0.5
        arparams (list): autoregressive lag  for cycle component, default [0.5]
        maparams (list): MA lags  for cycle component, default []
        sigma_eps (float): std. dev. of irregular componet, default 1.0
        sigma_eta (float): std. dev of trend disturbance, default 1.0
        sigma_zeta (float): std. dev of drift disturbance, default 1.0
        sigma_omega (float): std. dev of seasonal disturbance, default 1.0
        sigma_xi (float): std. dev of cycle component disturbance, default 1.0
        S_init (list): initial seasonal elements, of length s
    Output:
        sims (np.array): N X T array of simualted structural model time series
    '''
    #Check for valid beta_phi, must be < 1 and > -1
    if np.abs(phi_beta) >= 1.0:
        raise ValueError("Error: stochastic drift AR(1) parameter must have absolute va

    #Negate and add zero lag for AR and MA roots
    ar = np.r_[1, -np.array(ar_params)] # add zero-lag and negate
    ma = np.r_[1, np.array(ma_params)] # add zero-lag

    #Fit ARMA process, check invertibility
    arma_process = sm.tsa.ArmaProcess(ar, ma)
    if not arma_process.isstationary:
        raise ValueError("Error: invalid AR roots. Cycle component non-stationary.")

    #Check seasonal componet is appropriate length
    if S_init is None:
        S_init = np.zeros(S)
    else:
        if len(S_init) != S:
            raise ValueError("Initial values for seasonal component don't match frequen

    #Compute total length account for burn in
    sim_T = int((1+burn_in)*T*S)

    #And total length without burn in
    sim_T_nb = int(T*S)

    #Construct mean and covariance matrices for disturbances
    sigma_eps = 1/np.sqrt(T*S)
    sigma_eta = 1/np.sqrt(T*S)
    sigma_zeta = 1/np.sqrt(T*S)
    sigma_omega = 1/np.sqrt(T*S)
    sigma_xi = 1/np.sqrt(T*S)
    cov = np.diag([sigma_eps, sigma_eta, sigma_zeta, sigma_omega, sigma_xi])
    mean = np.zeros(5)

    #Draw from MVN distribution of disturbances
    dist = np.random.multivariate_normal(mean=mean, cov=cov, size=(N, sim_T))

    #Separate error terms from MVN draw, draw is shape N x sim_T x 5
    eps =  dist[:,:,0]
    eta = dist[:,:,1]
    zeta = dist[:,:,2]
    omega = dist[:,:,3]
    xi = dist[:,:,4]


    #Simulate beta(t). beta(1) is drawn from a dist, no need for burn in.
    #Initialize array of zeros, dim N x T-1
    beta_t = np.zeros((N,sim_T_nb-1))

    #Prepend a col of normal draws for beta_1
    beta_init_var = sigma_eta**2 / (1- phi_beta**2)
    beta_1 = np.random.normal(loc=0, scale=np.sqrt(beta_init_var), size=(N, 1))
    beta_t = np.hstack([beta_1, beta_t])

    #Iteratively compute beta_t
    for t in range(sim_T_nb-1):
        beta_t[:,t+1] = phi_beta*beta_t[:,t] + zeta[:,t]


    #The change in mu(t) is beta(t) + epsilon(t). Compute, then take cumulative sum for
    mu_t = beta_t + eta[:,0:sim_T_nb]
    mu_t = np.cumsum(mu_t, axis=1)

    #Use sm's ARIMA sim functionality for cycle
    psi_t = arma_process.generate_sample(nsample=(N,sim_T_nb), scale=sigma_eta, burnin=


    #Simulate seasonal component. Prepend supplied starting values.
    gamma_t = np.zeros((N, sim_T_nb-S))
    gamma_t = np.hstack([ np.array([S_init for n in range(N)]), gamma_t])
    for t in range(S-1, sim_T_nb-1):
        gamma_t[:,t+1] = gamma_t[:,  t-S+2:t].sum(axis=1) + omega[:,t]


    #Finally, construct y(t). Select last T*S obs to account for burn in.
    y_t = np.zeros((N, sim_T_nb))
    y_t = mu_t[:,-sim_T_nb:] + gamma_t[:,-sim_T_nb:] + psi_t[:,-sim_T_nb:] + eps[:,-sim

    return y_t

#Demonstrate
pd.DataFrame(sim_ss(S_init=[-1,1,0.5,0.8])[0:5,:]).T.plot()
plt.show()
```



Misc: CERRPCT

CERRPCT as an error measure is flawed.

$$\text{CERRPCT} \equiv \frac{\left(\sum_{t=1}^{T} \hat{Y}_t\right) - \left(\sum_{t=1}^{T} Y_t\right)}{\sum_{t=1}^{T} Y_t}$$

If $Y_t \geq 0$, this charitably measures the error between cumulative predicted and cumulative actual. Revenue forecasts over several quarters, for example, may show large errors in individual quarters but over the entire horizon be accurate. If a business plans around the horizon, and not individual quarters, a model with high quarterly errors but CERRPCT within tolerance is acceptable.

There are several issues.

1. There are an infinite number of choices for $\hat{Y}_1 \ldots \hat{Y}_{T-1}$ if $\hat{Y}_t \in R$. Only the last observation (or an arbitrary single observation) $\hat{Y}_T$ is uniquely determined to achieve CERRPCT of 0: $\hat{Y}_T = \sum_{t=1}^{T} Y_t - \sum_{t=1}^{T-1} \hat{Y}_t$. Comparing CERRPCT figures is meaningless.

1. Even with the restriction of non-negative values, $\hat{Y}_t \geq 0$, the number of ways to express the sum of actuals $\sum_{t=1}^{T} Y_t$ using T non-negative integers is ${}_{S+T-1}C_{T-1}$. This value ballons for large $S = \sum_{t=1}^{T} Y_t$.

1. Let $\hat{Y}_i = \sum_{t=1}^{T} Y_t$ for a single, arbitrary $i \in \{1 \ldots T\}$, and $\hat{Y}_t = 0$ otherwise. Error measures like MAPE, RMSE, etc., are high for large Y_t. If $\hat{Y}_t$ can be negative, true error measures go to infinity for arbitrarily large and offsettingly small individual forecast values such that they sum to 0.

Misc: Time Trend Regression

Consider the following model for X_t as a function of linear and quadratic time trends and a random walk:

$$\begin{aligned}
X_t &= \beta_0 t + \beta_1 t^2 + w_t \\
w_t &= w_{t-1} + e_t, e_t \sim \text{WN}(0, \sigma_u^2) \\
Y_t &\equiv \Delta X_t \\
Y_t &= \beta_0 - \beta_1 + 2\beta_1 t + e_t
\end{aligned}$$

Despite detrending, X_t is non-stationary because the disturbance term is a random walk. One estimation approach is to first-difference X_t, call it Y_t, and then regress Y_t on a linear time trend, and recover estimates $\hat{\beta}_1$ and $\hat{\beta}_0$, and standard errors using the Delta method. Y_t is trend stationary, and the regression works because the covariate is a deterministic linear time trend.

References

James G. MacKinnon, 2010. "Critical Values For Cointegration Tests," Working Paper 1227, Economics Department, Queen's University.

https://www.econ.queensu.ca/sites/econ.queensu.ca/files/wpaper/qed_wp_1227.pdf

Statsmodels, 2025. "statsmodels.tsa.stattools.adfuller", Documentation. Web.

https://www.statsmodels.org/dev/generated/statsmodels.tsa.stattools.adfuller.html